

Técnicas de diseño

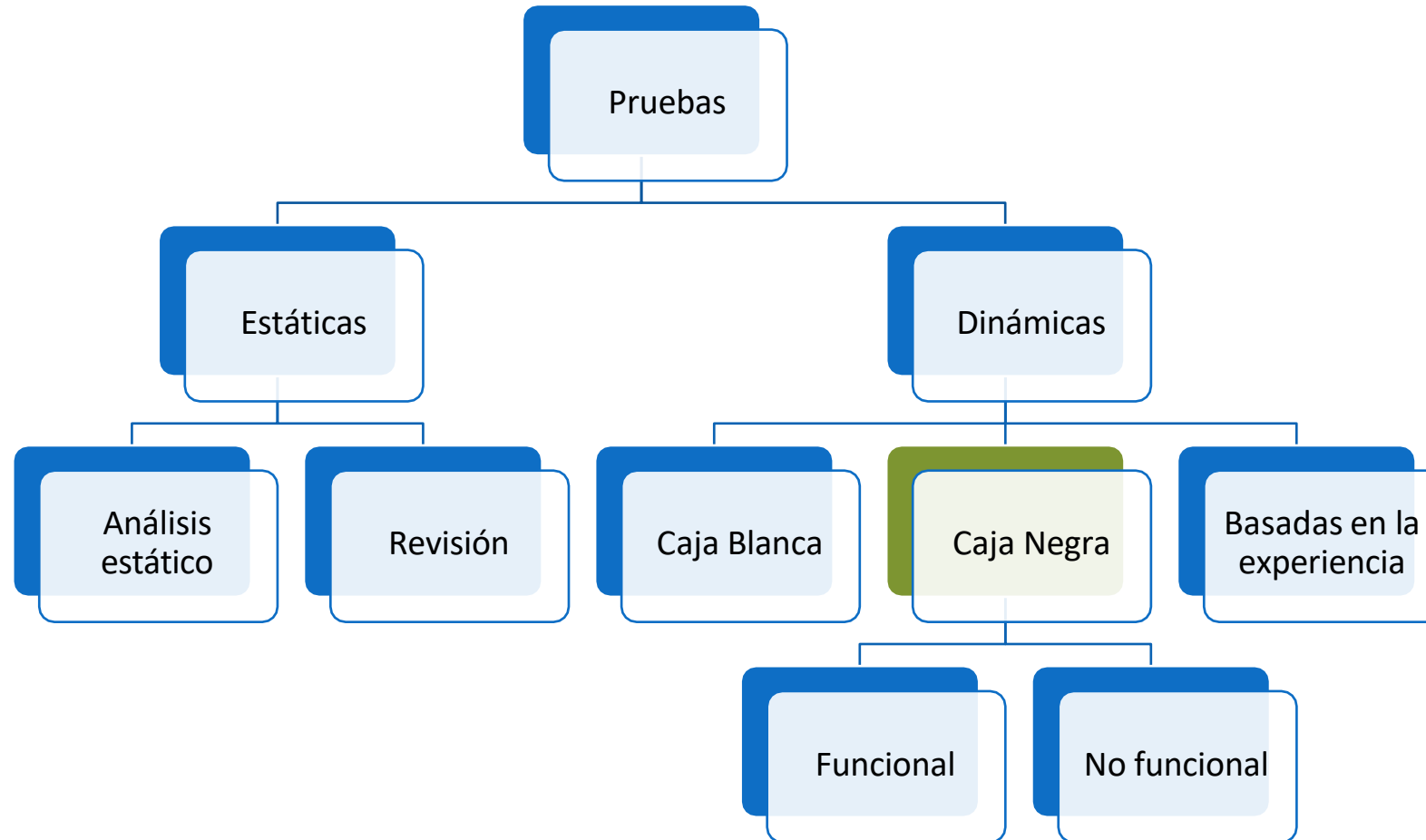
Pruebas de caja blanca

Aseguramiento de la calidad del software

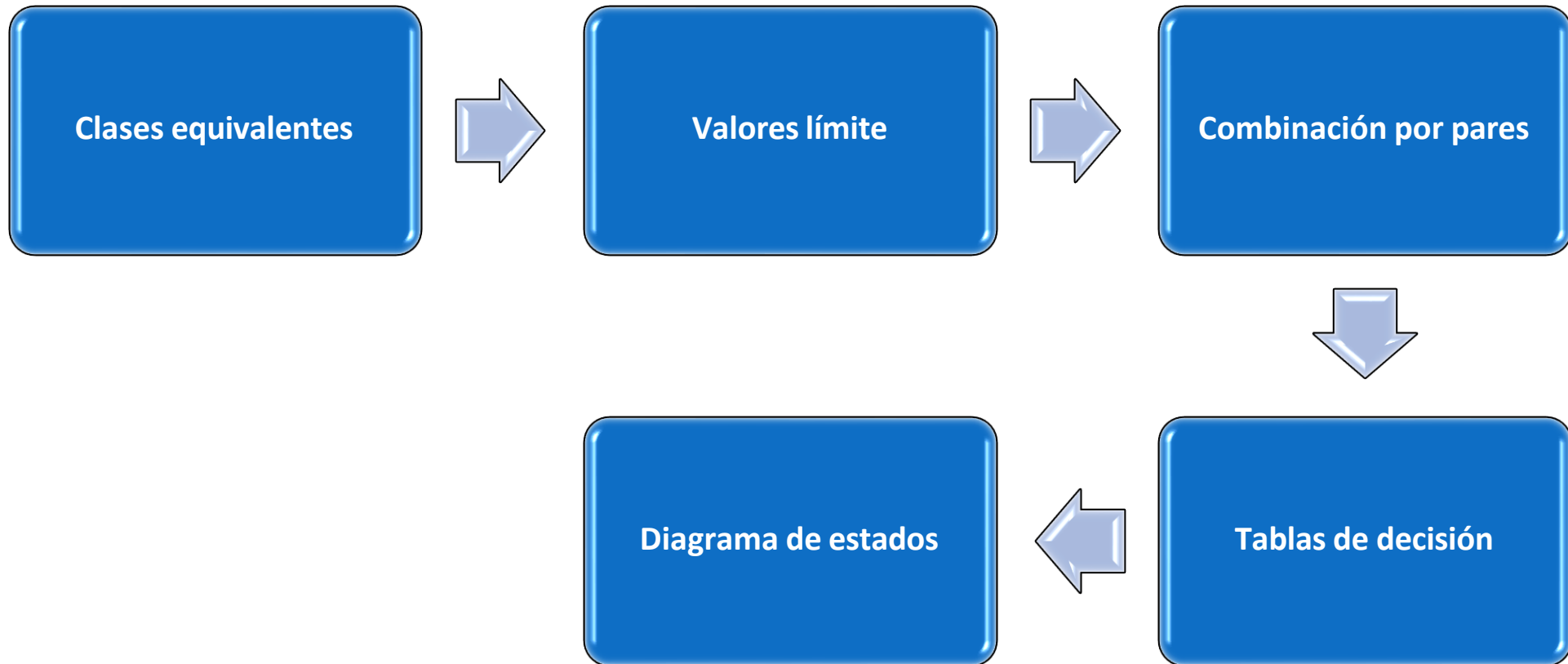
Contenidos

- Repaso
- Técnicas de diseño de pruebas
 - Prueba de sentencias
 - Prueba de decisión
 - Prueba de condición
 - Prueba de condiciones y decisiones
 - Prueba de cobertura modificada de condiciones/decisiones (MC/DC)
 - Prueba de múltiples condiciones
 - Prueba de caminos

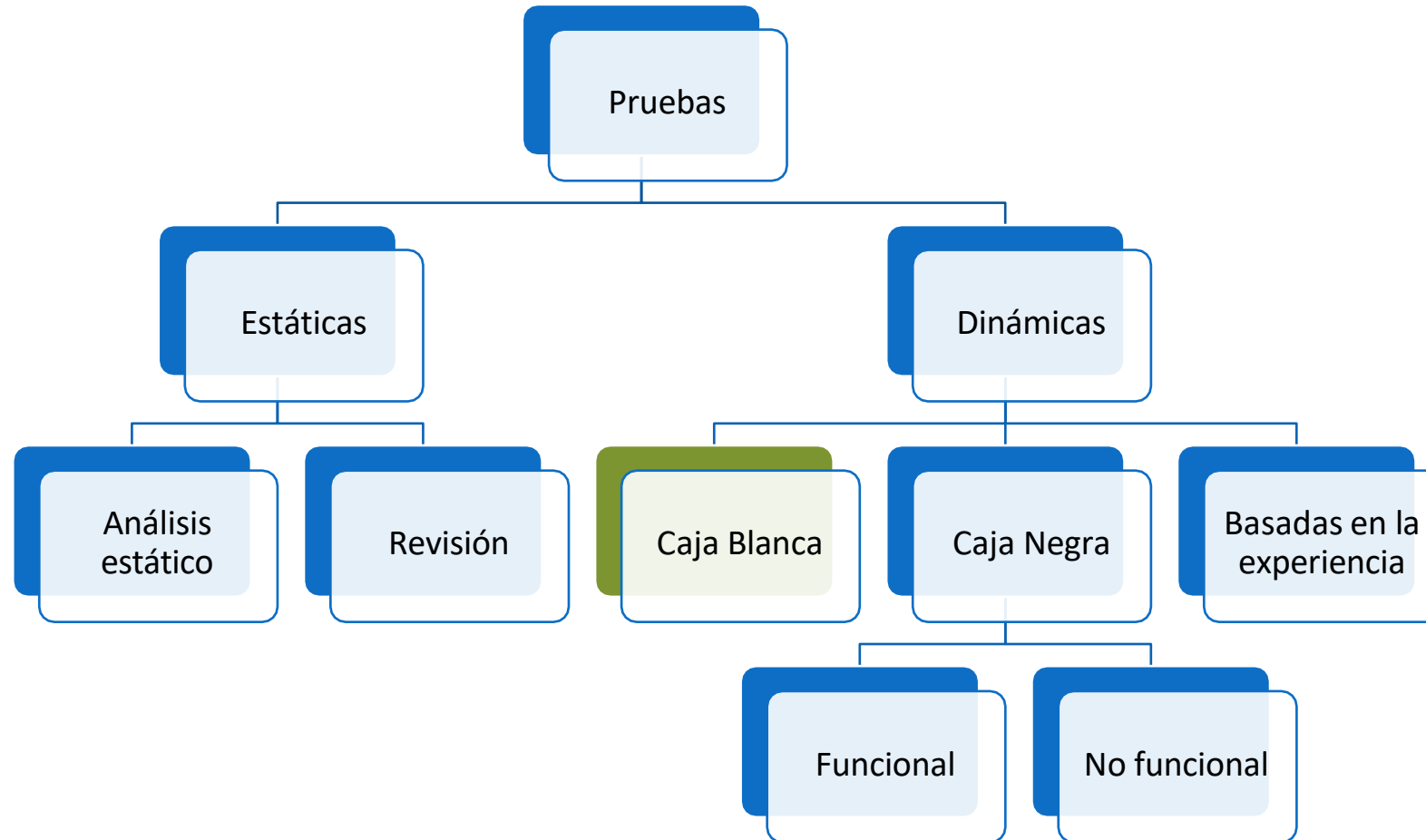
Tipos de pruebas



Caja Negra: Técnicas de diseño de pruebas



Tipos de pruebas



Caja Blanca

- Las pruebas basadas en la estructura se enfocan en **cómo trabaja el sistema internamente**
 - Determinan y alcanzan un **nivel de cobertura de flujos de control** basada en el análisis de código
 - Determinan y alcanzan un **nivel de cobertura de flujos de datos** basada en el análisis de código
 - Determinan y alcanzan un **nivel de cobertura de interfaces, clases, flujos de llamadas** y similares basada en el análisis de APIs, diseño del sistema, etc.
- La cobertura de la estructura es una manera de comprobar las brechas en las pruebas basadas en la especificación y experiencia

Pruebas de
software:
Caja Blanca

Prueba de sentencias

Prueba de decisión

Prueba de condición

Prueba de condiciones y decisiones

Prueba de cobertura modificada de condiciones/decisiones
(MC/DC)

Prueba de múltiples condiciones

Prueba de caminos

Pruebas de
software:
Caja Blanca

Prueba de sentencias

Prueba de decisión
Prueba de decisión

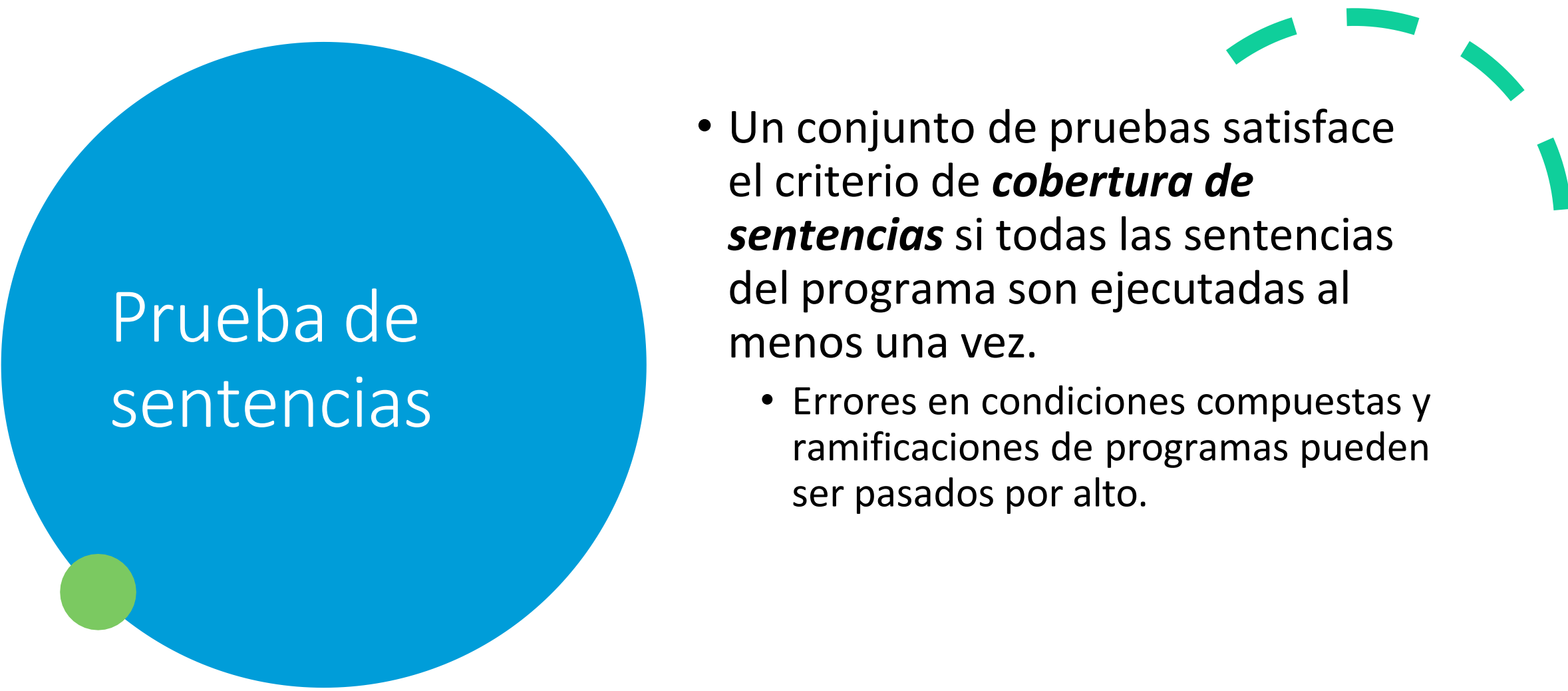
Prueba de condición
Prueba de condición

Prueba de condiciones y decisiones

Prueba de cobertura modificada de condiciones/decisiones
(MC/DC)

Prueba de múltiples condiciones

Prueba de caminos



Prueba de sentencias

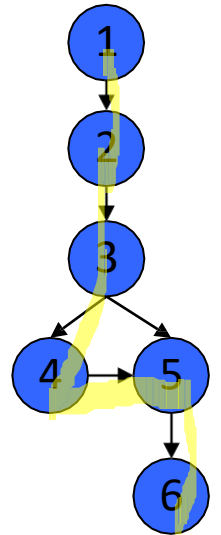
- Un conjunto de pruebas satisface el criterio de ***cobertura de sentencias*** si todas las sentencias del programa son ejecutadas al menos una vez.
 - Errores en condiciones compuestas y ramificaciones de programas pueden ser pasados por alto.

Prueba de sentencias

- Ejemplo

- El siguiente programa determina si un año dado, es o no bisiesto:

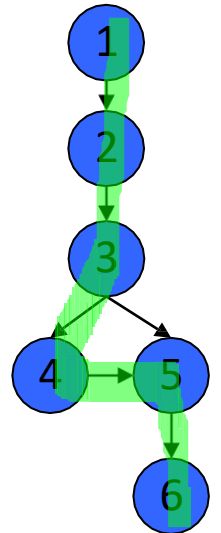
```
1. bool bisiesto(int a) {  
2.   bool b = false;  
3.   if ((a%4==0) && (a%100!= 0))  
4.     b = true;  
5.   return b;  
6. }
```



Prueba de sentencias

- Ejemplo
 - El siguiente programa determina si un año dado, es o no bisiesto:

```
1. bool bisiesto(int a) {  
2.   bool b = false;  
3.   if ((a%4==0) && (a%100!= 0))  
4.     b = true;  
5.   return b;  
6. }
```



Pruebas de
software:
Caja Blanca

Prueba de sentencias

Prueba de decisión
Prueba de decisión

Prueba de condición
Prueba de condición

Prueba de condiciones y decisiones

Prueba de cobertura modificada de condiciones/decisiones
(MC/DC)

Prueba de múltiples condiciones

Prueba de caminos

Prueba de decisión

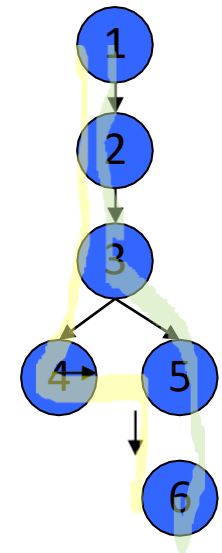
- Una **decisión** es un *punto en el código en el que se produce una ramificación o bifurcación*.
 - Ej.: condiciones de ciclos, condiciones de if-then-else
 - La decisión puede ser una simple expresión booleana
 $A > B$
 - O bien, podría ser arbitrariamente compleja
 $(A > B) \ || \ (X + Y == -1) \ \&\& \ (D != \text{true})$
- Un caso de prueba satisface el criterio de cobertura de decisión *si todas las decisiones del programa son ejecutadas por true y por false al menos una vez*.
- **Propiedad:** *cobertura de **decisión** es más fuerte que cobertura de sentencias*.
 - Es decir, si un caso de prueba satisface cobertura de decisión, también satisface cobertura de sentencias

Prueba de decisión

- Ejemplo

- El siguiente programa determina si un año dado, es o no bisiesto:

```
1. bool bisiesto(int a) {  
2.   bool b = false;  
3.   if ((a%4==0) && (a%100!= 0))  
4.     b = true;  
5.   return b;  
6. }
```

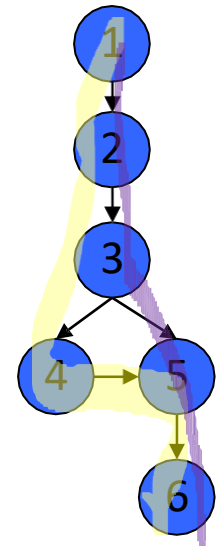


Prueba de decisión

- Ejemplo

- El siguiente programa determina si un año dado, es o no bisiesto:

```
1. bool bisiesto(int a) {  
2.   bool b = false;  
3.   if ((a%4==0) && (a%100!= 0))  
4.     b = true;  
5.   return b;  
6. }
```



Pruebas de
software:
Caja Blanca

Prueba de sentencias

Prueba de decisión
Prueba de decisión

Prueba de condición
Prueba de condición

Prueba de condiciones y decisiones

Prueba de cobertura modificada de condiciones/decisiones
(MC/DC)

Prueba de múltiples condiciones

Prueba de caminos

Prueba de condición

- Las pruebas de condiciones consideran las **condiciones atómicas**
- Un caso de prueba satisface el criterio de cobertura de condición *si cada condición atómica de cada decisión es ejecutada por true y por false al menos una vez.*
- ¡No es lo mismo que todas las combinaciones!
- **OJO:** *cobertura de **condición** NO es más fuerte que cobertura de decisión.*

Condición atómica

- Una **decisión** puede estar compuesta por una o más condiciones
 - Ejemplos
 - $x \rightarrow \text{verdadero}$
 - Hay una condición atómica
 - $D \ \&\& \ F \rightarrow \text{falso}$
 - Hay dos condiciones atómicas
 - $(A \ || \ B) \ \&\& \ (C == D) \rightarrow \text{verdadero}$
 - Hay tres condiciones atómicas
 - $(a > b) \ || \ (x+y == -1) \ \&\& \ (d != \text{true}) \rightarrow \text{verdadero}$
 - Hay tres condiciones atómicas
 - En cada caso, hay todavía una sola decisión para toda la expresión

+

•

o

Prueba de condición

- En la página anterior, x, D, F, A y B todas deben ser probadas con **verdadero y falso** en las pruebas
 - $(C == D)$ debe ser probada con V y F
 - $(a > b)$ debe ser probada con V y F
 - $(x+y == -1)$ debe ser probada con V y F
 - $(d != \text{true})$ debe ser probada con V y F

Prueba de condición

- Ejemplo
 - Consideremos el siguiente programa

```
bool find(int[] list, int size, int val) {  
    int index = 0;  
    bool found = False;  
    while (index < (size) && !found) {  
        if (list[index] == val)  
            found = True;  
        index++;  
    }  
    return found;  
}
```

- ¿Qué entradas deberíamos dar para conseguir cobertura de condición?

Prueba de condición

- Ejemplo
 - Consideremos el siguiente programa
- ¿Qué entradas deberíamos dar para conseguir cobertura de condición?
- {list[1,2,3] y val=4, list[1,2,3] y val=3}

```
bool find(int[] list, int size, int val) {  
    int index = 0;  
    bool found = False;  
    while (index < (size) && !found) {  
        if (list[index] == val)  
            found = True;  
        index++;  
    }  
    return found;  
}
```

¿Son las
pruebas de
condiciones
suficientemente
buenas?

- Considere el siguiente fragmento de código:
 - if (a and b) then
- La cobertura de condiciones dice que cada condición atómica debe ser probada en ambos sentidos:
 - $a == F, b == V \rightarrow \text{FALSO}$
 - $a == V, b == F \rightarrow \text{FALSO}$
- ¿Se tiene cobertura de decisiones?
 - Se necesita un caso de prueba más
 - $a == V, b == V \rightarrow \text{VERDADERO}$

Ejercicios

Dado el siguiente fragmento de código:

```
print ("A");  
if (a<1)  
    then print ("B");  
endif  
print ("C")  
if (b<2)  
    then print ("D");  
endif  
print ("E");
```

Calcule la cantidad de casos de prueba mínimos necesarios para obtener una **cobertura de sentencia** del 100%.

Nota: Los valores de a y b son arbitrarios e independientes.

Ejercicios

Dado el siguiente fragmento de código:

```
print ("A");  
if (a<1)  
    then print ("B");  
endif  
print ("C")  
if (b<2)  
    then print ("D");  
endif  
print ("E");
```

Calcule la cantidad de casos de prueba mínimos necesarios para obtener una **cobertura de decisión** del 100%.

Nota: Los valores de a y b son arbitrarios e independientes.

Pruebas de
software:
Caja Blanca

Prueba de sentencias

Prueba de decisión
Prueba de decisión

Prueba de condición
Prueba de condición

Prueba de condiciones y decisiones

Prueba de cobertura modificada de condiciones/decisiones
(MC/DC)

Prueba de múltiples condiciones

Prueba de caminos

Prueba de condiciones y decisiones

- Puede ser que no se necesiten pruebas adicionales más allá de la cobertura de condiciones
- Limitaciones/dificultades: a veces necesitan más pruebas que el nivel de decisión, lo que puede agravar la presión de tiempo

Más fuerte que sólo la condición o decisión

- Una decisión puede ser una simple expresión booleana
 - $a > b$
- O bien, podría ser arbitrariamente compleja
 - $(a > b) \parallel (x + y == -1) \&\& (d != \text{true})$
- Para lograr una cobertura de decisiones, sólo alguna vez necesitamos dos casos de prueba; una prueba donde la decisión se evalúa como verdadera y falsa

Prueba de condiciones y decisiones

Cada condición atómica debe ser evaluada en una prueba en ambos sentidos (V / F) y la cobertura de decisiones también se debe satisfacer

Derivación de las pruebas: identificar datos para ambos sentidos y añadir pruebas para garantizar la cobertura de decisiones

Hipótesis de defectos: sólo probando todas las condiciones puede no ser suficiente

Prueba de condiciones y decisiones

- Ejemplo
- Considere el siguiente fragmento de código:
 - If (a and b) then ...
- La cobertura de condiciones se cumple con:
 - $a == \text{false} \wedge b == \text{true} \rightarrow \text{false}$
 - $a == \text{true} \wedge b == \text{false} \rightarrow \text{false}$
- La cobertura de condiciones y decisiones se logra con la adición de
 - $a == \text{true} \wedge b == \text{true} \rightarrow \text{true}$
- *¿Son estas pruebas suficientes?*

Ejercicio

Considere el siguiente segmento de código:

```
if ((tax > 0) && (income < 250000)){  
    refund = true;  
}  
else{  
    refund = false;  
}
```

Se pide

- Definir los casos de prueba mínimo necesarios para obtener una cobertura de condición y decisión.

Ejercicio

Considere el siguiente segmento de código:

```
if ((tax > 0) && (income < 250000)){  
  refund = true;  
}  
else{  
  refund = false;  
}
```

CP1 = tax = 1000 = T
income = 30000 ~~T~~

D = V

CP2 = tax = -10 F
income = 300000 F

D = F

Se pide

- Definir los casos de prueba mínimo necesarios para obtener una cobertura de condición y decisión.



Ejercicio

Utilice este fragmento de código de Delphi:

```
If ((DSourceNode.StringData2 = 'Test')  
    and (DDTgtNode, StringData2='Test2')  
    and (DDCtrlPressed or DDShiftPressed)  
    and (DDSourceNode.Parent = DDTgtNode.Parent)  
    and (NOT DoingDuplicate)) then begin
```

```
Raise TstBudExcept.Create('No puedes copiar una prueba a su  
propio padre');
```

```
End;
```

Se pide

- Diseñar pruebas para lograr una cobertura del 100% de Decisión/Condición.

Ejercicio

Utilice este fragmento de código de Delphi:

```
If ((DSourceNode.StringData2 = 'Test')  
    and (DDTgtNode, StringData2='Test2')  
    and (DDCtrlPressed or DDShiftPressed)  
    and (DDSourceNode.Parent = DDTgtNode.Parent)  
    and (NOT DoingDuplicate)) then begin
```

Raise TstBudExcept.Create('No puedes copiar una prueba a su propio padre');

End;

	C1	C2	C3	C4	C5	C6	D
CP1 =	T	T	T	T	T	F	T
CP2 =	F	F	F	F	F	T	F

Se pide

- Diseñar pruebas para lograr una cobertura del 100% de Decisión/Condición.

Pruebas de
software:
Caja Blanca

Prueba de sentencias

Prueba de decisión
Prueba de decisión

Prueba de condición
Prueba de condición

Prueba de condiciones y decisiones

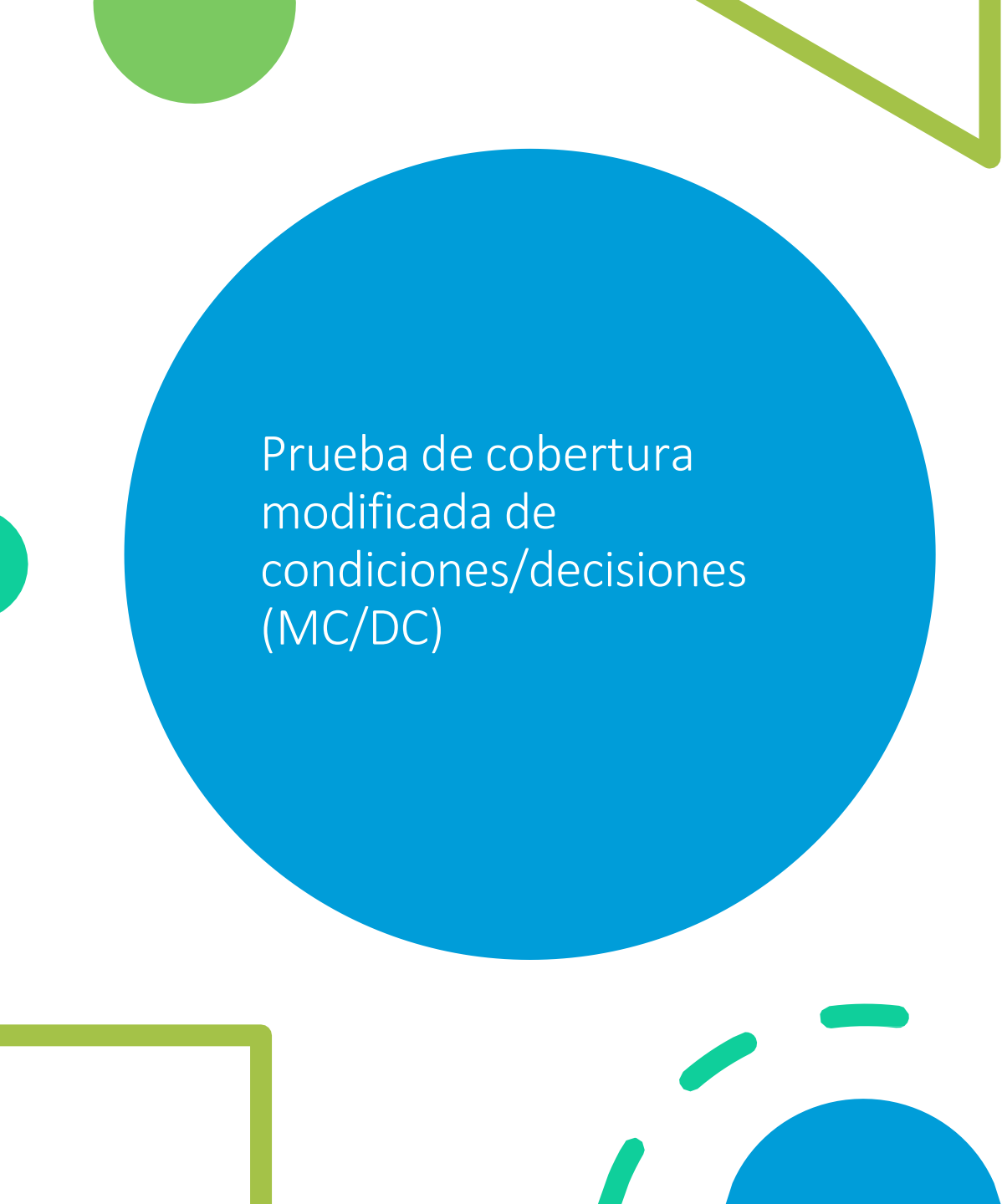
Prueba de cobertura modificada de condiciones/decisiones
(MC/DC)

Prueba de múltiples condiciones

Prueba de caminos

Prueba de cobertura
modificada de
condiciones/decisiones
(MC/DC)

- MC/DC es la cobertura de condiciones y decisiones. Además, de por lo menos una prueba en la que cada condición atómica controla el resultado de la decisión.
 - Como regla general, ***dado N condiciones atómicas, se requiere $N+1$ pruebas***
 - Limitaciones/dificultades: no siempre alcanzable cuando las condiciones están acopladas o cuando el compilador hace circuitos cortos
- 

A decorative graphic in the top-left corner consisting of a green circle and a green line forming a right-angled triangle. Another green line forms a right-angled triangle in the bottom-left corner. A blue circle is partially visible at the bottom center.

Prueba de cobertura modificada de condiciones/decisiones (MC/DC)

- Concepto: cada condición atómica debe ser evaluada en una prueba en ambos sentidos (V/F) y la cobertura de decisiones también debe ser satisfecha y cada condición determina el resultado, una vez con Verdadero y otra vez con Falso
- Criterios de cobertura: asegúrate de que cada condición provoca al menos un resultado V y uno F
- Hipótesis de los defectos: sólo probando todas las decisiones y las condiciones puede no ser suficiente

Proceso MC/DC sistemático

Ejemplo: if (a or b)

1. Dibujar una tabla con 3 columnas, con la expresión lógica y los dos posibles resultados. Añadir una fila con espacios en blanco para cada condición, con el número de espacios blancos igual al número total de condiciones
2. Llenar los espacios en blanco en diagonal en la segunda columna con "T" y en la tercera columna con "F"
3. Reemplazar los espacios en blanco restantes con valores neutros (F para OR, T para AND)
4. Eliminar cualquier prueba duplicada de las celdas

Proceso MC/DC sistemático

Ejemplo: if (a or b)

1. Dibujar una tabla con 3 columnas, con la expresión lógica y los dos posibles resultados. Añadir una fila con espacios en blanco para cada condición, con el número de espacios blancos igual al número total de condiciones
2. Llenar los espacios en blanco en diagonal en la segunda columna con "T" y en la tercera columna con "F"
3. Reemplazar los espacios en blanco restantes con valores neutros (F para OR, T para AND)
4. Eliminar cualquier prueba duplicada de las celdas

a OR b	T	F
a	--	--
b	--	--

Proceso MC/DC sistemático

Ejemplo: if (a or b)

1. Dibujar una tabla con 3 columnas, con la expresión lógica y los dos posibles resultados. Añadir una fila con espacios en blanco para cada condición, con el número de espacios blancos igual al número total de condiciones
2. Llenar los espacios en blanco en diagonal en la segunda columna con "T" y en la tercera columna con "F"
3. Reemplazar los espacios en blanco restantes con valores neutros (F para OR, T para AND)
4. Eliminar cualquier prueba duplicada de las celdas

a OR b	T	F
a	<u>T</u> _	<u>F</u> _
b	_ <u>T</u>	_ <u>F</u>

Proceso MC/DC sistemático

Ejemplo: if (a or b)

1. Dibujar una tabla con 3 columnas, con la expresión lógica y los dos posibles resultados. Añadir una fila con espacios en blanco para cada condición, con el número de espacios blancos igual al número total de condiciones
2. Llenar los espacios en blanco en diagonal en la segunda columna con "T" y en la tercera columna con "F"
3. Reemplazar los espacios en blanco restantes con valores neutros (F para OR, T para AND)
4. Eliminar cualquier prueba duplicada de las celdas

a OR b	T	F
a	<u>T</u> <u>F</u>	<u>F</u> <u>F</u>
b	<u>F</u> <u>T</u>	<u>F</u> <u>F</u>

Proceso MC/DC sistemático

Ejemplo: if (a or b)

1. Dibujar una tabla con 3 columnas, con la expresión lógica y los dos posibles resultados. Añadir una fila con espacios en blanco para cada condición, con el número de espacios blancos igual al número total de condiciones
2. Llenar los espacios en blanco en diagonal en la segunda columna con "T" y en la tercera columna con "F"
3. Reemplazar los espacios en blanco restantes con valores neutros (F para OR, T para AND)
4. Eliminar cualquier prueba duplicada de las celdas

a OR b	T	F
a	<u>T</u> <u>F</u>	<u>F</u> <u>F</u> 
b	<u>F</u> <u>T</u>	<u>F</u> <u>F</u>

Ejercicio

Considere el siguiente segmento de código:

```
if ((tax > 0) && (income < 250000)){  
  refund = true;  
}  
else{  
  refund = false;  
}
```

Se pide

- Definir los casos de prueba mínimo necesarios para obtener una cobertura de condición modificada y decisión MC/DC.



Ejercicio

Considere el siguiente segmento de código:

```
if ((tax > 0) && (income < 250000)){
```

```
    refund = true;
```

```
}
```

```
else{
```

```
    refund = false;
```

```
}
```

$a \text{ and } b$	T	F
a	T T	<u>F T</u>
b	T T	T F

Se pide

- Definir los casos de prueba mínimo necesarios para obtener una cobertura de condición modificada y decisión MC/DC.

Pruebas de
software:
Caja Blanca

Prueba de sentencias

Prueba de decisión
Prueba de decisión

Prueba de condición
Prueba de condición

Prueba de condiciones y decisiones

Prueba de cobertura modificada de condiciones/decisiones
(MC/DC)

Prueba de múltiples condiciones

Prueba de caminos

Prueba de múltiples condiciones

- Probar todas las combinaciones posibles de las condiciones en una decisión
- **Requiere 2^n pruebas**, donde n es el número de condiciones atómicas no acopladas
- La evaluación de un circuito corto generalmente reducirá el número de pruebas
- Limitaciones/dificultades: fácil de determinar las pruebas (cobertura completa de la tabla de verdad), pero difícil de ejecutar las pruebas (debido al volumen)

Prueba de múltiples condiciones



Probar exhaustivamente cada combinación de condiciones que pueda afectar a un resultado



Criterios de cobertura: $(\# \text{ combinaciones de condiciones ejecutadas} / \# \text{ total de combinaciones de condiciones}) = 100\%$



Hipótesis de los defectos: ¡los defectos podrían estar en cualquier lugar!

Prueba de múltiples condiciones

- Ejemplo – Dado el siguiente fragmento de código:
 - If ((a OR b) and C) then
 - ...
- Cree una tabla de verdad para todas las permutaciones a, b y c
 - $2^n = 2^3 = 8$
- Cree un caso de prueba por cada combinación

a	b	c
TRUE	TRUE	TRUE
TRUE	TRUE	FALSE
TRUE	FALSE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	TRUE
FALSE	TRUE	FALSE
FALSE	FALSE	TRUE
FALSE	FALSE	FALSE

+

•

○

Prueba de múltiples condiciones

- Circuito corto
 - Para algunos lenguajes de programación, los compiladores generan código que dejan de evaluar una expresión booleana una vez que el valor final se conoce:
 - Si la primera condición atómica en (a AND b) es falsa, no hay necesidad de evaluar la segunda
 - Si la primera condición atómica en (a OR b) es verdadera, no hay necesidad de evaluar la segunda
 - Para la mayoría de estos lenguajes de programación, la evaluación es de ~~derecha a izquierda~~, aunque el paréntesis puede influir en la forma de funciona
 - La evaluación de circuito corto puede influir en el número de pruebas reales necesario para la cobertura MC/DC

Prueba de múltiples condiciones

Circuito corto – Ejemplo

if ((a OR b) AND c) then ...

- Evalué la expresión de izquierda a derecha
- Detenga la evaluación tan pronto como el valor de la expresión es conocido (circuito corto)
- Considerar los paréntesis
- Crear las pruebas para las combinaciones de circuito corto (es decir, aquellas combinaciones que pueden realmente afectar el resultado)

a	b	c	Resultado
TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE
TRUE	FALSE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE

Prueba de múltiples condiciones

Circuito corto – Ejemplo

if ((a OR b) AND c) then ...

- Evalué la expresión de izquierda a derecha
- Detenga la evaluación tan pronto como el valor de la expresión es conocido (circuito corto)
- Considerar los paréntesis
- Crear las pruebas para las combinaciones de circuito corto (es decir, aquellas combinaciones que pueden realmente afectar el resultado)

a	b	c	Resultado
TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE
TRUE	FALSE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE

Prueba de múltiples condiciones

Circuito corto – Ejemplo

if ((a OR b) AND c) then ...

- Evalué la expresión de izquierda a derecha
- Detenga la evaluación tan pronto como el valor de la expresión es conocido (circuito corto)
- Considerar los paréntesis
- Crear las pruebas para las combinaciones de circuito corto (es decir, aquellas combinaciones que pueden realmente afectar el resultado)

a	b	c	Resultado
TRUE	-	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE
TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE

Prueba de múltiples condiciones

Circuito corto – Ejemplo

if ((a OR b) AND c) then ...

- Evalué la expresión de izquierda a derecha
- Detenga la evaluación tan pronto como el valor de la expresión es conocido (circuito corto)
- Considerar los paréntesis
- Crear las pruebas para las combinaciones de circuito corto (es decir, aquellas combinaciones que pueden realmente afectar el resultado)

a	b	c	Resultado
TRUE	-	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE
TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE

Prueba de múltiples condiciones

Circuito corto – Ejemplo

if ((a OR b) AND c) then ...

- Evalué la expresión de izquierda a derecha
- Detenga la evaluación tan pronto como el valor de la expresión es conocido (circuito corto)
- Considerar los paréntesis
- Crear las pruebas para las combinaciones de circuito corto (es decir, aquellas combinaciones que pueden realmente afectar el resultado)

a	b	c	Resultado
TRUE	-	TRUE	TRUE
TRUE	-	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE

Prueba de múltiples condiciones

Circuito corto – Ejemplo

if ((a OR b) AND c) then ...

- Evalué la expresión de izquierda a derecha
- Detenga la evaluación tan pronto como el valor de la expresión es conocido (circuito corto)
- Considerar los paréntesis
- Crear las pruebas para las combinaciones de circuito corto (es decir, aquellas combinaciones que pueden realmente afectar el resultado)

a	b	c	Resultado
TRUE	-	TRUE	TRUE
TRUE	-	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE

Prueba de múltiples condiciones

Circuito corto – Ejemplo

if ((a OR b) AND c) then ...

- Evalué la expresión de izquierda a derecha
- Detenga la evaluación tan pronto como el valor de la expresión es conocido (circuito corto)
- Considerar los paréntesis
- Crear las pruebas para las combinaciones de circuito corto (es decir, aquellas combinaciones que pueden realmente afectar el resultado)

a	b	c	Resultado
TRUE	-	TRUE	TRUE
TRUE	-	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	-	FALSE

Ejercicio

Considere el siguiente segmento de código:

```
if ((tax > 0) && (income < 250000)){  
    refund = true;  
}  
else{  
    refund = false;  
}
```

Se pide

- Definir los casos de prueba mínimo necesarios para obtener una cobertura de condiciones múltiples.

Ejercicio

Considere el siguiente segmento de código:

```
if ((tax > 0) && (income < 250000)){  
    refund = true;  
}  
else{  
    refund = false;  
}
```

$$2^n = 2^2 = 4$$

a	b
T	T
T	F
F	T
F	F

Se pide

- Definir los casos de prueba mínimo necesarios para obtener una cobertura de condiciones múltiples.

Ejercicio

- $2^n = 2^6 = 64$

	A	B	C	D	E	F	R
1.							
2.							
3.							
4.							
5.							
6.							
7.							
8.							
9.							
10.							
11.							
12.							
13.							
14.							
15.							
16.							
17.							
18.							
19.							
20.							
21.							
22.							
23.							
24.							
25.							
26.							
27.							
28.							
29.							
30.							
31.							
32.							

	A	B	C	D	E	F	R
33.							
34.							
35.							
36.							
37.							
38.							
39.							
40.							
41.							
42.							
43.							
44.							
45.							
46.							
47.							
48.							
49.							
50.							
51.							
52.							
53.							
54.							
55.							
56.							
57.							
58.							
59.							
60.							
61.							
62.							
63.							
64.							

Ejercicio

- $2^n = 2^6 = 64$

	A	B	C	D	E	F	R
1.	T	T	T	T	T	T	
2.	T	T	T	T	T	F	
3.	T	T	T	T	F	T	
4.	T	T	T	T	F	F	
5.	T	T	T	F	T	T	
6.	T	T	T	F	T	F	
7.	T	T	T	F	F	T	
8.	T	T	T	F	F	F	
9.	T	T	F	T	T	T	
10.	T	T	F	T	T	F	
11.	T	T	F	T	F	T	
12.	T	T	F	T	F	F	
13.	T	T	F	F	T	T	
14.	T	T	F	F	T	F	
15.	T	T	F	F	F	T	
16.	T	T	F	F	F	F	
17.	T	F	T	T	T	T	
18.	T	F	T	T	T	F	
19.	T	F	T	T	F	T	
20.	T	F	T	T	F	F	
21.	T	F	T	F	T	T	
22.	T	F	T	F	T	F	
23.	T	F	T	F	F	T	
24.	T	F	T	F	F	F	
25.	T	F	F	T	T	T	
26.	T	F	F	T	T	F	
27.	T	F	F	T	F	T	
28.	T	F	F	T	F	F	
29.	T	F	F	F	T	T	
30.	T	F	F	F	T	F	
31.	T	F	F	F	F	T	
32.	T	F	F	F	F	F	

	A	B	C	D	E	F	R
33.	F	T	T	T	T	T	
34.	F	T	T	T	T	F	
35.	F	T	T	T	F	T	
36.	F	T	T	T	F	F	
37.	F	T	T	F	T	T	
38.	F	T	T	F	T	F	
39.	F	T	T	F	F	T	
40.	F	T	T	F	F	F	
41.	F	T	F	T	T	T	
42.	F	T	F	T	T	F	
43.	F	T	F	T	F	T	
44.	F	T	F	T	F	F	
45.	F	T	F	F	T	T	
46.	F	T	F	F	T	F	
47.	F	T	F	F	F	T	
48.	F	T	F	F	F	F	
49.	F	F	T	T	T	T	
50.	F	F	T	T	T	F	
51.	F	F	T	T	F	T	
52.	F	F	T	T	F	F	
53.	F	F	T	F	T	T	
54.	F	F	T	F	T	F	
55.	F	F	T	F	F	T	
56.	F	F	T	F	F	F	
57.	F	F	F	T	T	T	
58.	F	F	F	T	T	F	
59.	F	F	F	T	F	T	
60.	F	F	F	T	F	F	
61.	F	F	F	F	T	T	
62.	F	F	F	F	T	F	
63.	F	F	F	F	F	T	
64.	F	F	F	F	F	F	

Pruebas de
software:
Caja Blanca

Prueba de sentencias

Prueba de decisión
Prueba de decisión

Prueba de condición
Prueba de condición

Prueba de condiciones y decisiones

Prueba de cobertura modificada de condiciones/decisiones
(MC/DC)

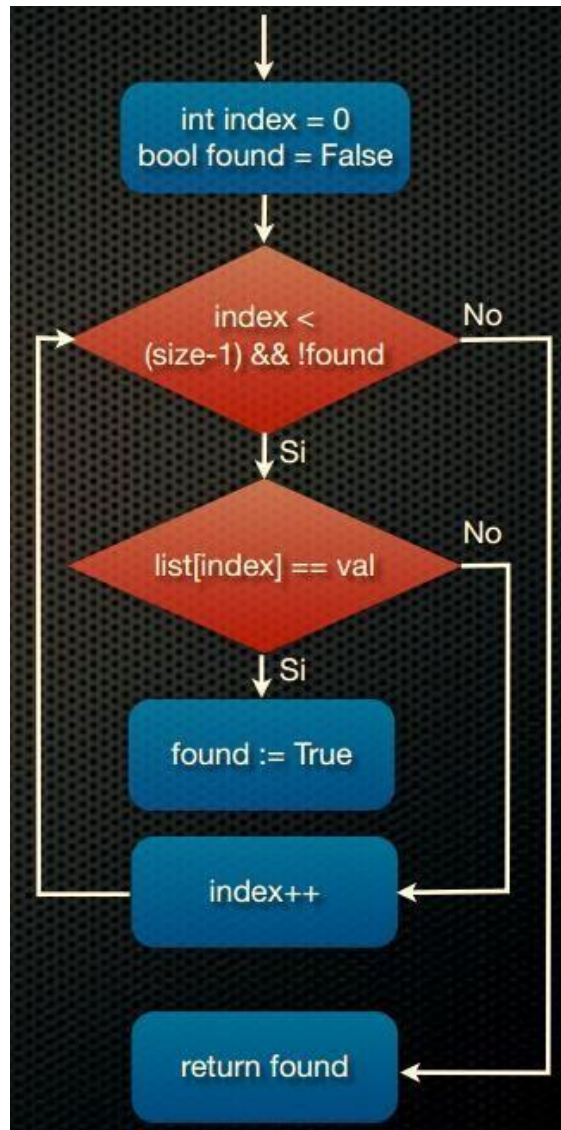
Prueba de múltiples condiciones

Prueba de caminos

Prueba de caminos

- Un conjunto de casos de prueba satisface el criterio de cobertura de caminos *si todos los caminos del grafo de flujo de control del programa son recorridos al menos una vez.*
 - Es un **criterio muy fuerte**: conseguirlo puede requerir suites muy grandes.
- *Suelen imponerse restricciones* al criterio para hacerlo practicable:
 - **cobertura de caminos simples**: requiere cubrir *caminos sin repetición de arcos o aristas.*
 - **cobertura de caminos elementales**: requiere cubrir caminos *sin repetición de nodos.*



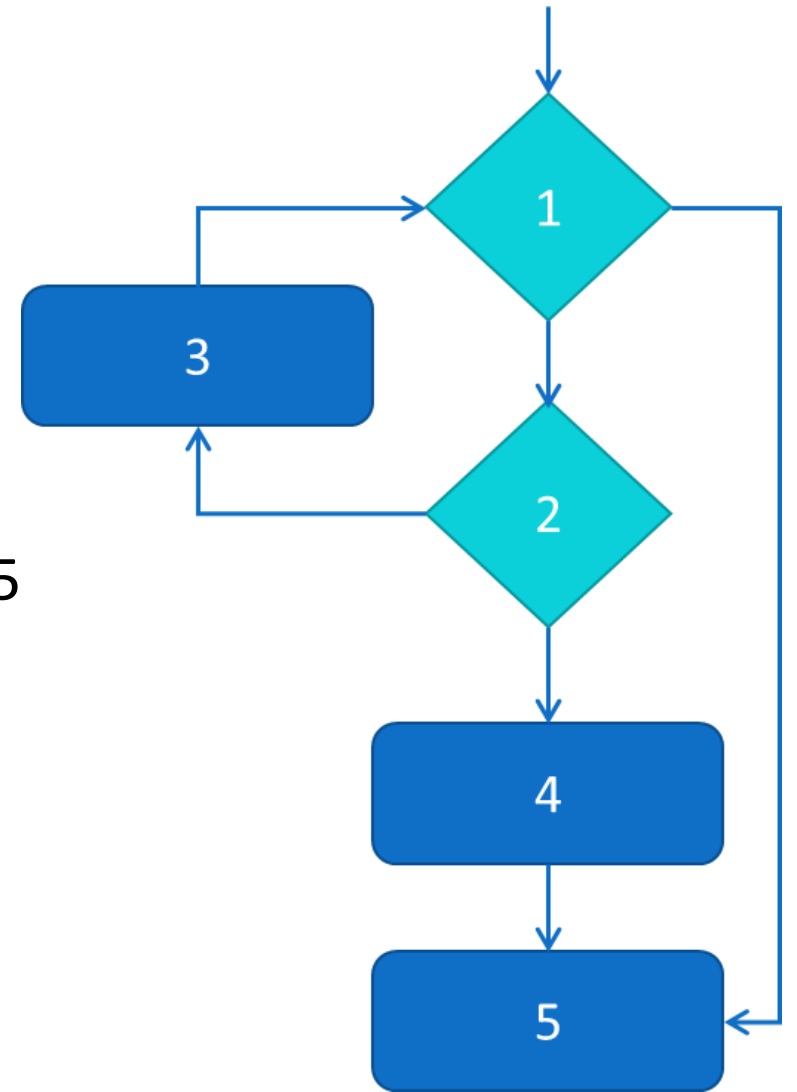


Prueba de caminos

- Grafo de flujo de control
 - Es una **representación**, mediante grafos dirigidos, **del flujo de control del programa**:
 - Los **nodos** del grafo representan *segmentos de sentencias* que se ejecutan secuencialmente.
 - Las **decisiones** representan *bifurcaciones*.
 - Los **arcos** del grafo representan *transferencias de control entre nodos*. Son las líneas que unen dos nodos.

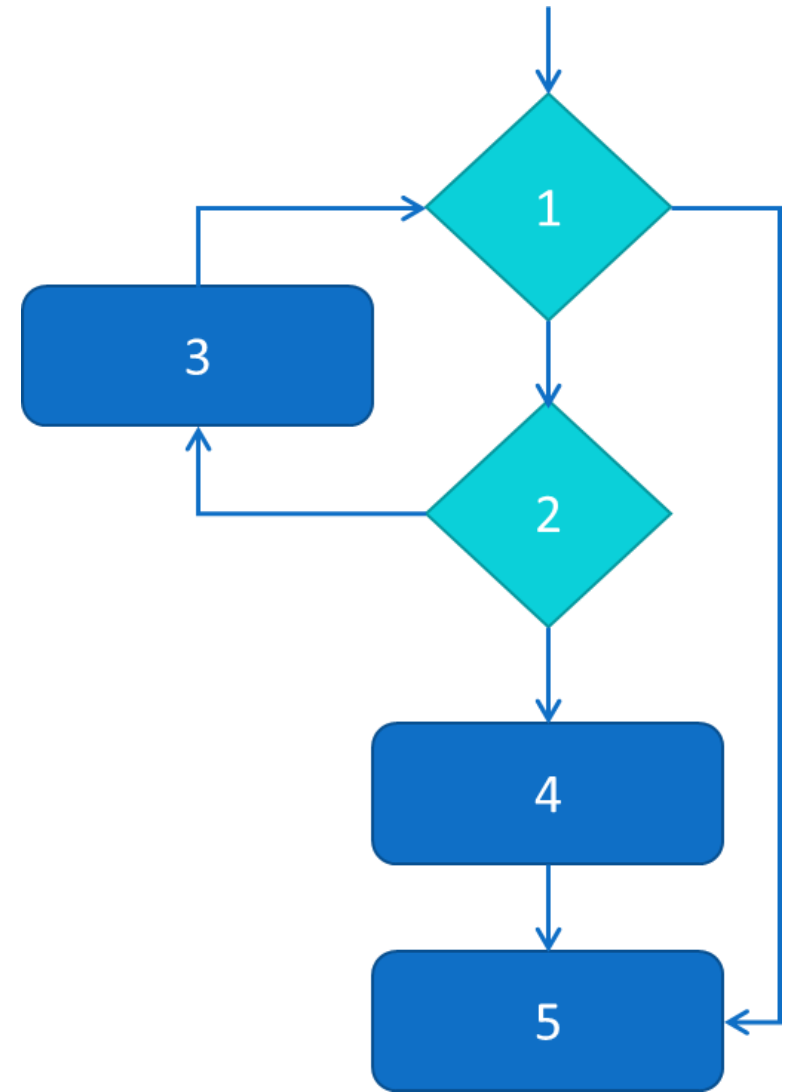
Prueba de caminos

- Cobertura de caminos
 - Caminos posibles
 - 1, 5
 - 1,2,4,5
 - 1,2,3,1,2,4,5
 - 1,2,3,1,2,3,1,2,4,5
 - 1,2,3,1,2,3,1,2,3,1,2,4,5
 - ...
 - 1,2,3,1,5
 - 1,2,3,1,2,3,1,5
 - 1,2,3,1,2,3,1,2,3,1,5
 - ...



Prueba de caminos

- Cobertura de caminos
 - Caminos simples:
 - 1, 5
 - 1,2,4,5
 - 1,2,3,1,5
 - Caminos elementales:
 - 1,5
 - 1,2,4,5
 - 1,2,3,1,5
 - 1,2,3,1,2,4,5

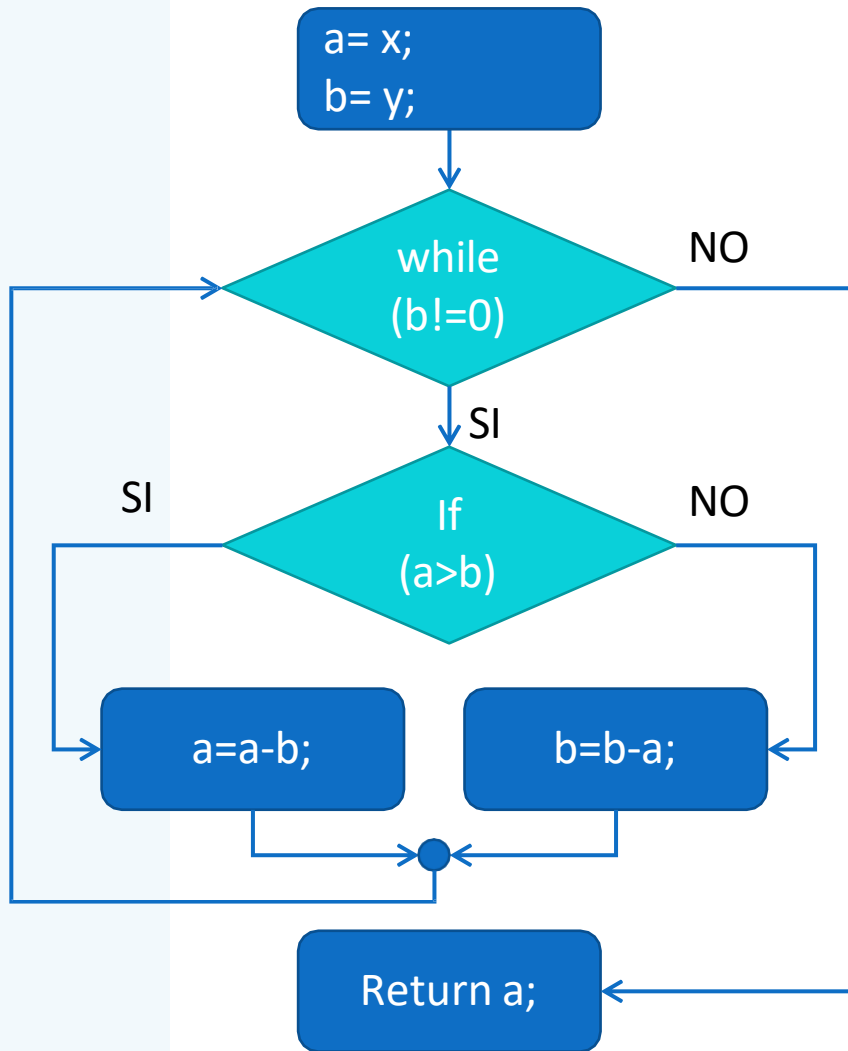


Prueba de caminos

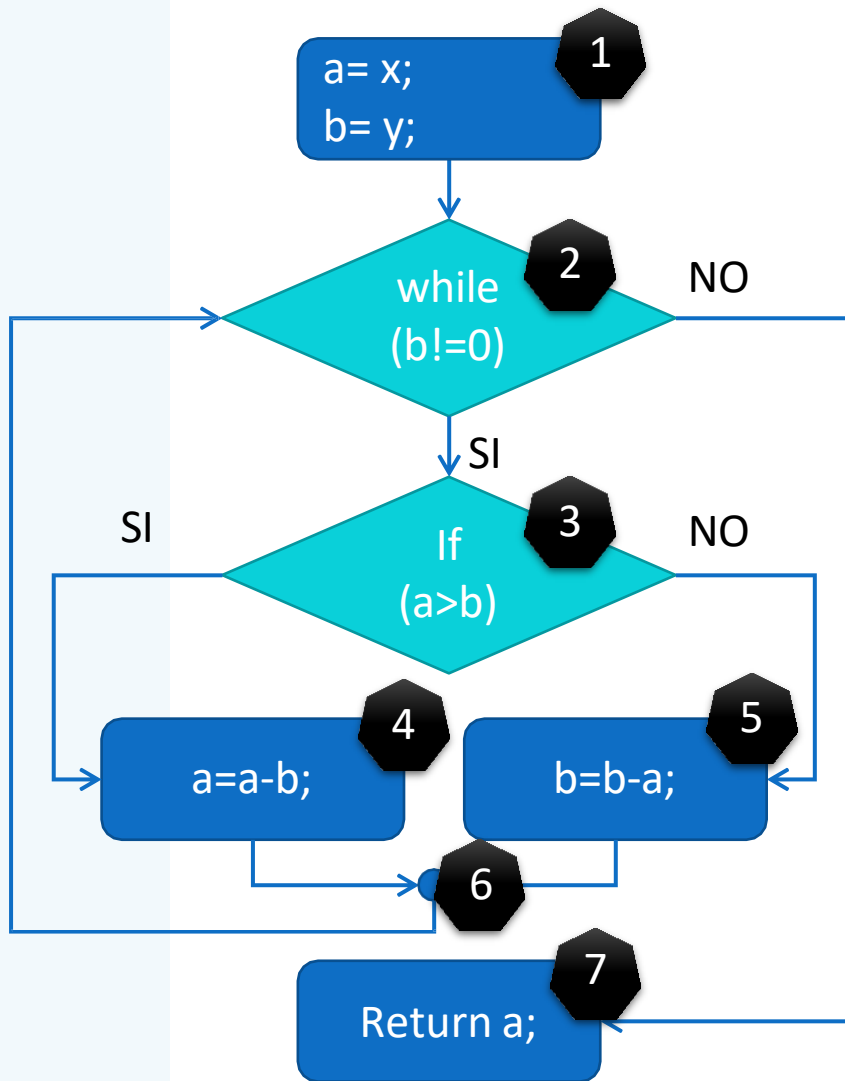
- Ejemplo
 - Consideremos el siguiente programa:
- ¿Qué entradas deberíamos dar para conseguir cobertura de caminos simple?

```
int mcd(int x, int y) {  
    int a= x;  
    int b= y;  
    while (b!=0) {  
        if (a>b)  
            a=a-b;  
        else  
            b=b-a;  
    }  
    return a;  
}
```

Prueba de caminos



Prueba de caminos

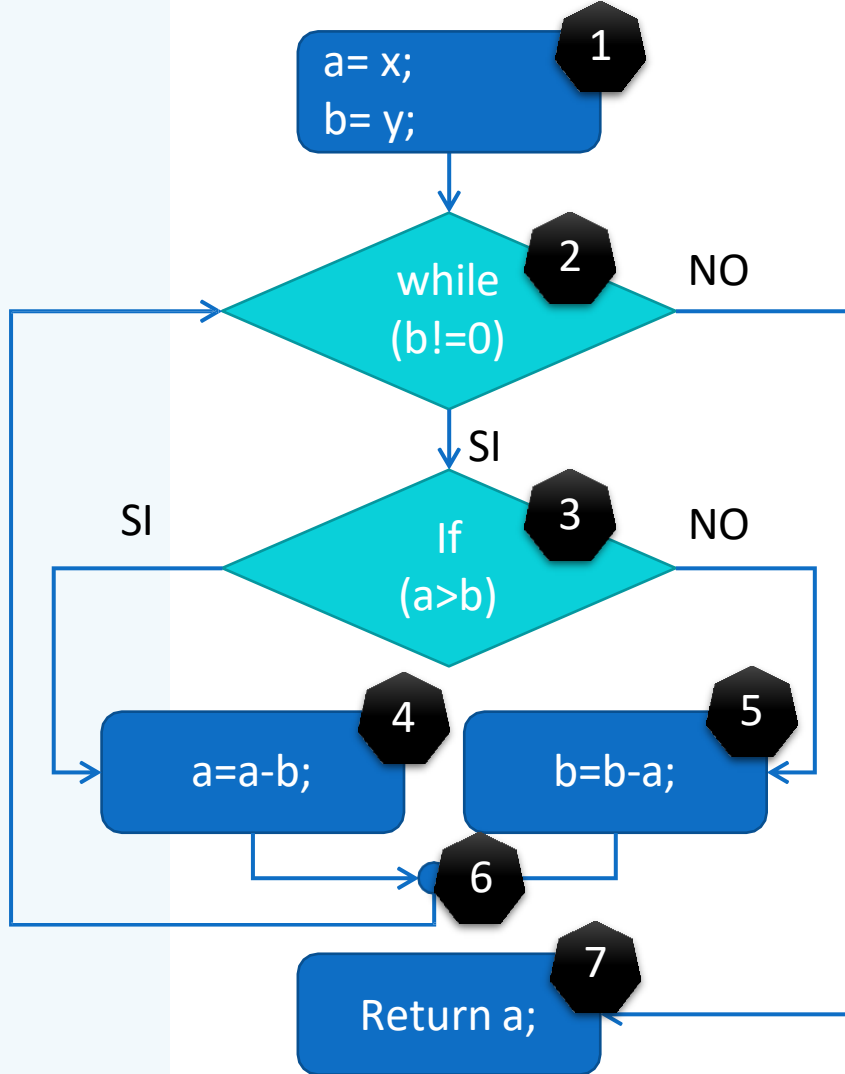


Prueba de caminos

- Cobertura de caminos

- Caminos simples:

- 1,2,7
 - 1,2,3,4,6,2,7
 - 1,2,3,5,6,2,7



Prueba de caminos

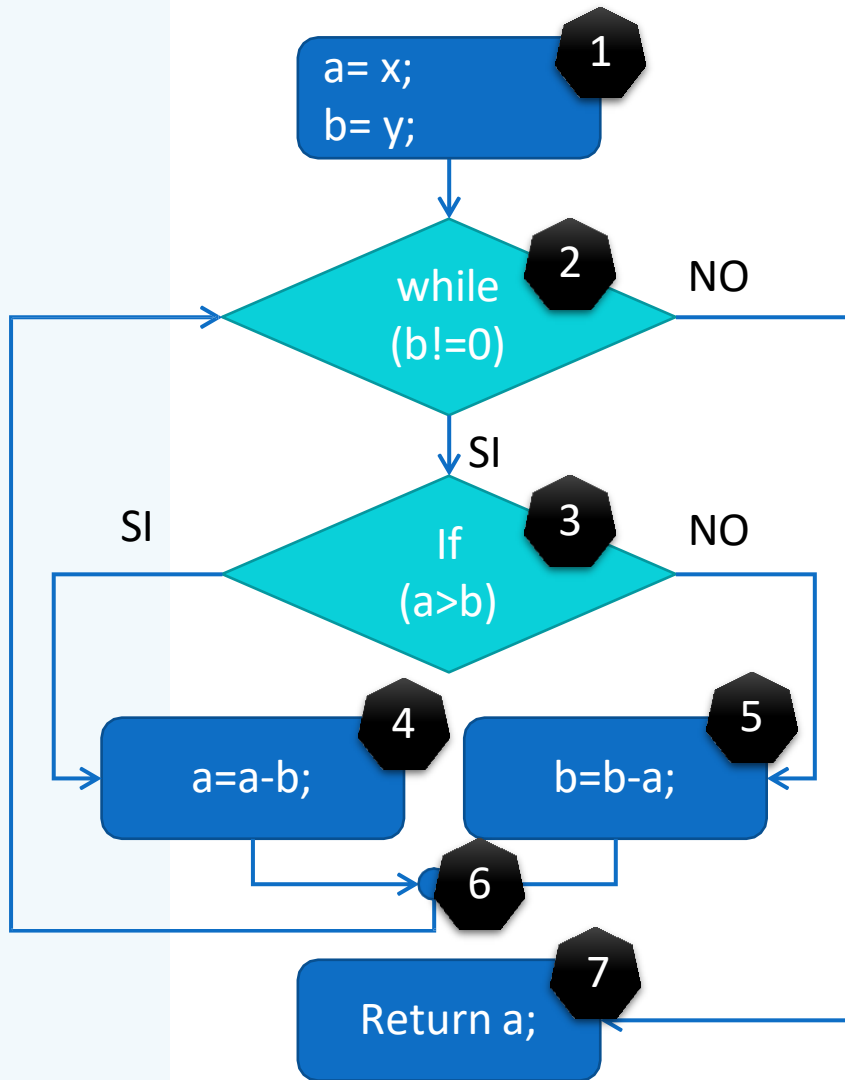
- Cobertura de caminos

- Caminos simples:

- 1,2,7
 - 1,2,3,4,6,2,7
 - 1,2,3,5,6,2,7

- Casos de prueba

- $\{\{a=1, b=0\}, \{a=1, b=1\}, \{a=2, b=1\}\}$





Prueba de caminos

- Complejidad ciclomática (CC)
 - Es una *métrica* que estima la **complejidad lógica** de un componente de software.
 - Es un *indicador* del **número de caminos independientes** que existen en un grafo.
 - Con un punto de entrada y un punto de salida, $CC = \text{número de decisiones} + 1$.
 - CC sirve como *cota superior* del número de test necesario para *cobertura de decisiones*.
 - *Estadísticamente*: mayor CC implica mayor número de defectos.

Prueba de caminos

- Complejidad ciclomática (CC)

- Formas de calcular

- $V(G) = a - n + 2$

- $V(G) = r$

- $V(G) = d + 1$

Donde

- a : # de arcos o aristas del grafo.

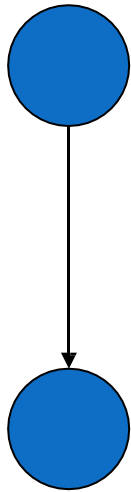
- n : # de nodos.

- r : # de regiones cerradas del grafo.

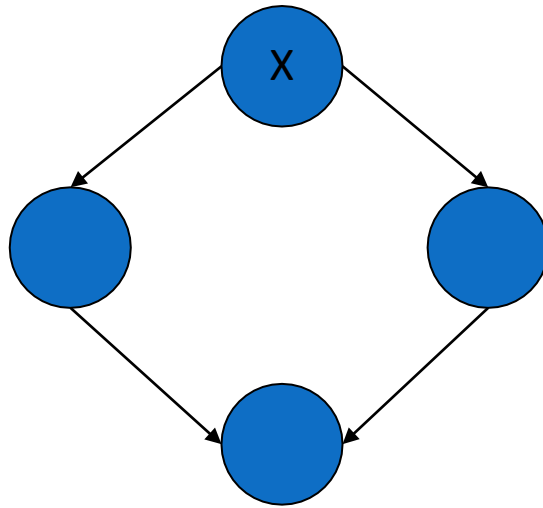
- d : # de decisiones

Prueba de caminos

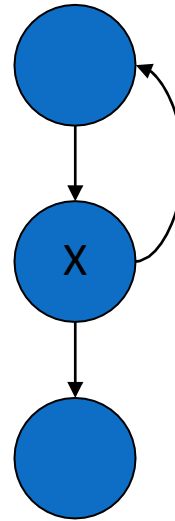
Estructuras



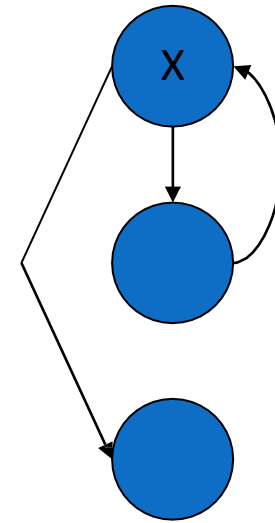
Secuencia



Si x Entonces...
(If x Then ... Else...)



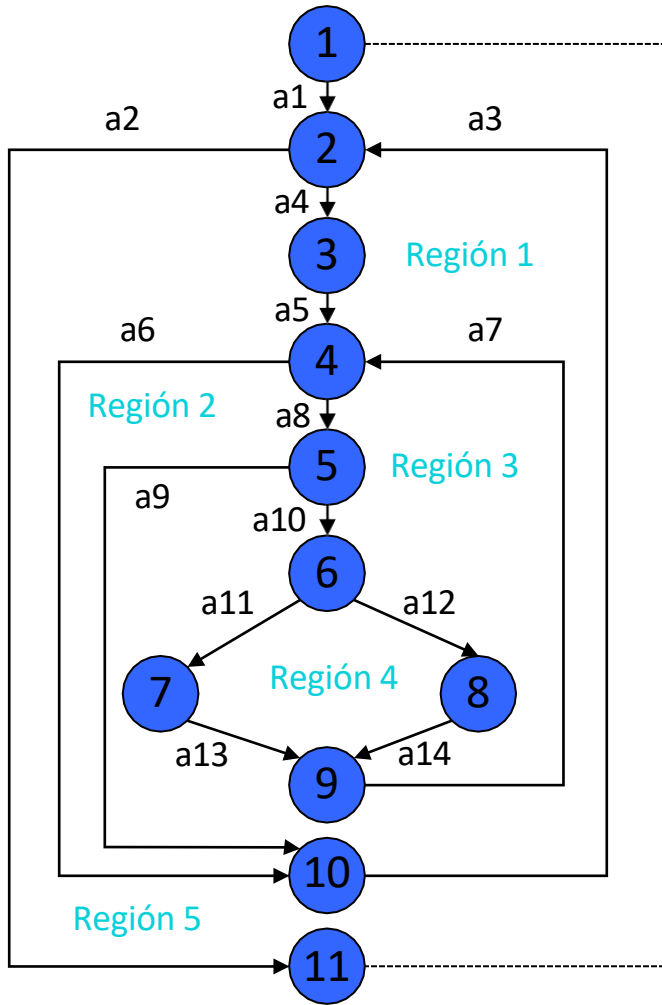
Hacer ... hasta x
(Do ... Until x)
Repetir



Mientras x Hacer ...
(While x Do ...)

Prueba de caminos

- $V(G) = 14 - 11 + 2 = 5$
- $V(G) = 5$ Regiones Cerradas.
- $V(G) = 4 + 1 = 5$ Condiciones





Prueba de caminos

- Complejidad ciclomática (CC)
 - Pasos para obtención de casos de prueba
 - Dibujar el grafo de flujo
 - Calcular la complejidad ciclomática
 - Determinar un conjunto básico
 - Preparar los casos de prueba para el camino básico

Prueba de caminos

- Cobertura de caminos

1 Begin

2 Lee DatosPersona

3 CodigoBarras = CodigoBarras + 1

4 If Edad >= 30 and Persona = "VIP" then

5 If Cargo ="Director" then

6 ColorTarjeta = "Dorado"

7 else

8 Color Tarjeta = "Rojo"

9 endif

10 Else

11 ColorTarjeta = "Blanco"

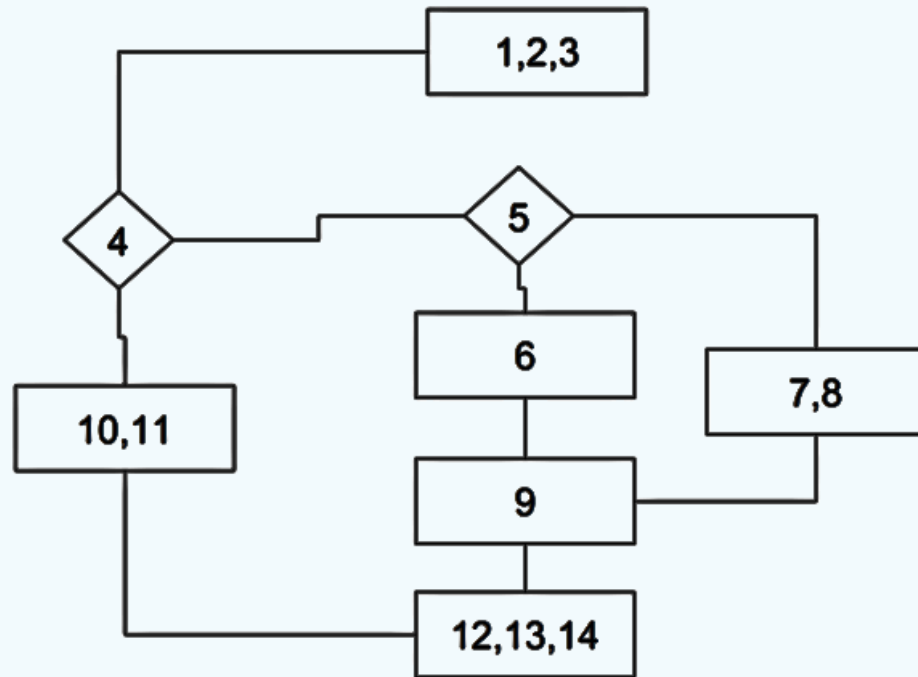
12 Endif

13 PrintTarjeta CodigoBarras, Nombre, Fnacimiento,
ColorTarjeta

14 End

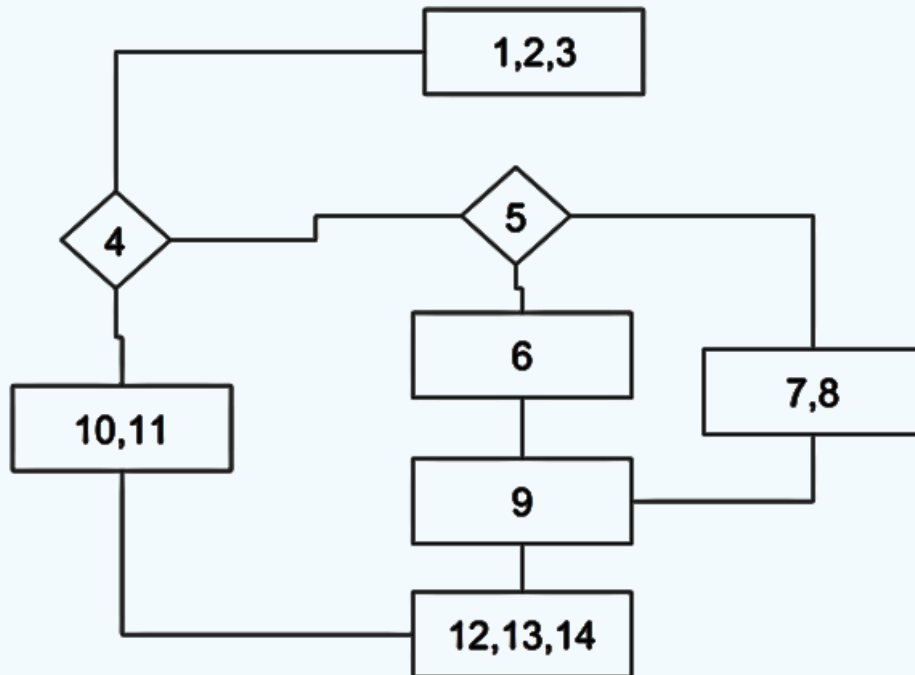
Prueba de caminos

- Cobertura de caminos



Prueba de caminos

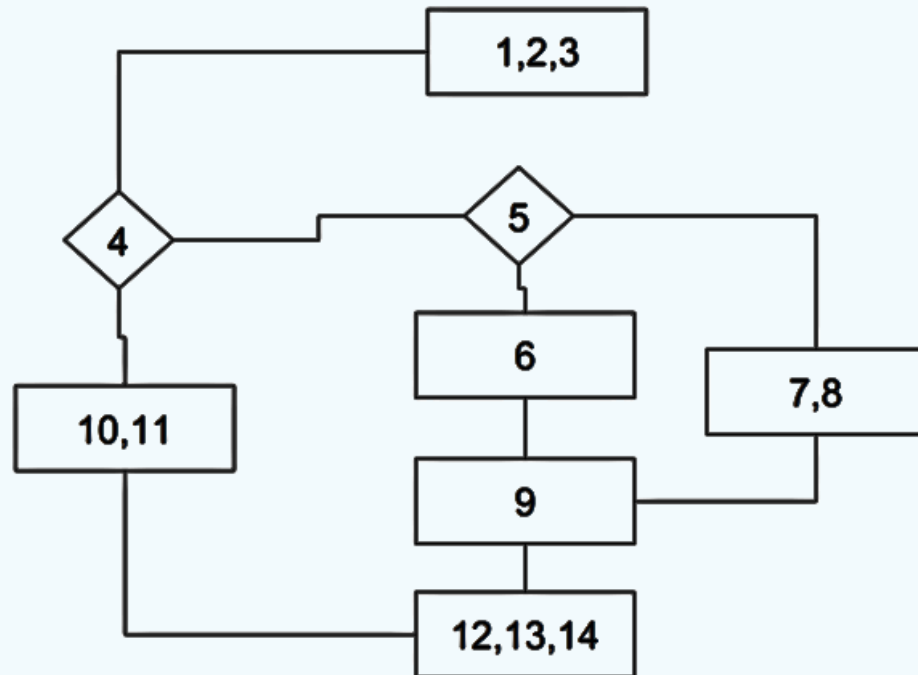
- Cobertura de caminos



- $A - N + 2 = 9 - 8 + 2 = 3$
- $N^{\circ} \text{ Regiones} + 1 = 3$
- Caminos independientes:
 - 1,2,3,4,10,11,12,13,14
 - 1,2,3,4,5,7,8,9,12,13,14
 - 1,2,3,4,5,6,9,12,13,14

Prueba de caminos

- Cobertura de caminos



- $A - N + 2 = 9 - 8 + 2 = 3$
- $N^{\circ} \text{ Regiones} + 1 = 3$
- Caminos independientes:
 - 1,2,3,4,10,11,12,13,14
 - 1,2,3,4,5,7,8,9,12,13,14
 - 1,2,3,4,5,6,9,12,13,14
- Casos de prueba:
 - Edad = 35, Persona = "VIP", Cargo = "Director"
 - Edad = 35, Persona = "VIP", Cargo = "Secretario"
 - Edad = 25, Persona = "VIP", Cargo = "Director"

Prueba de caminos

- Thomas McCabe, establece en sus trabajos los siguientes valores de referencia para la complejidad ciclomática:
 - 1 - 10, métodos sencillos, sin mucho riesgo.
 - 11 - 20, métodos medianamente complejos, con riesgo moderado.
 - 21 - 50, métodos complejos, con alto riesgo.
 - > 50, métodos inestables, de altísimo riesgo.

Ejercicios

```
void classify (int n) {  
    if(n > 0)  
        printf("Positivo\n");  
    if(n < 0)  
        printf("Negativo\n");  
    if(n == 0)  
        printf("Cero\n");  
}
```

$$cc = a - n + 2 = 9 + 7 + 2 = 4$$

$$cc = d + 1 = 3 + 1 = 4$$

$$cc = r = 4$$

Ejercicios

$$c_1 = 135F$$

$$c_2 = 1235F$$

$$c_3 = 1345F$$

$$c_4 = 1356F$$

$$n = -5$$

$$n = 10$$

$$n = 0$$

```
void classify (int n) {
```

```
1 if(n > 0)
```

```
2 printf("Positivo\n");
```

```
3 if(n < 0)
```

```
4 printf("Negativo\n");
```

```
5 if(n == 0)
```

```
6 printf("Cero\n");
```

```
}
```

```
7
```



Ejercicio



Ejercicios

```
int search(char s[], int ch) {  
    int i, fDone, fFound;  
    i = fDone = fFound = 0;  
    while (!fDone){  
        if s([i] == ch){  
            fDone = 1;  
            fFound = 1;  
        }  
        else if (s[1] == '\0')  
            fDone = 1;  
        i++;  
    }  
    return fFound;  
}
```

$$c = a - n + 2 = 10 - 8 + 2 = 4$$

$$c = d + 1 = 3 + 1 = 4$$

$$c = r = 4$$

Ejercicios

$$c_1 = 1 \ 2 \ 8$$

$$c_2 = 1 \ 2 \ 3 \ 4 \ 7 \ 2 \ 8$$

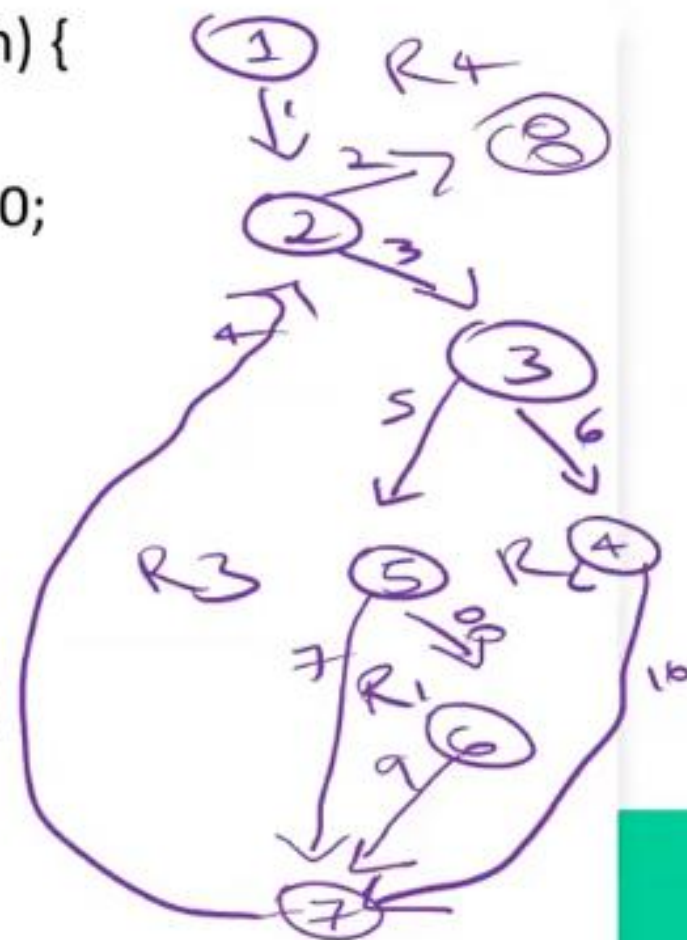
$$c_3 = 1 \ 2 \ 3 \ 5 \ 7 \ 2 \ 8$$

$$c_4 = 1 \ 2 \ 3 \ 5 \ 6 \ 7 \ 2 \ 8$$

```

int search(char s[], int ch) {
1  int i, fDone, fFound;
   i = fDone = fFound = 0;
2  while (!fDone){
3    if (s[i] == ch){
4      fDone = 1;
      fFound = 1;
    }
5    else if (s[i] == '\0'){
6      fDone = 1;
7    i++;
    }
8  return fFound;
}

```

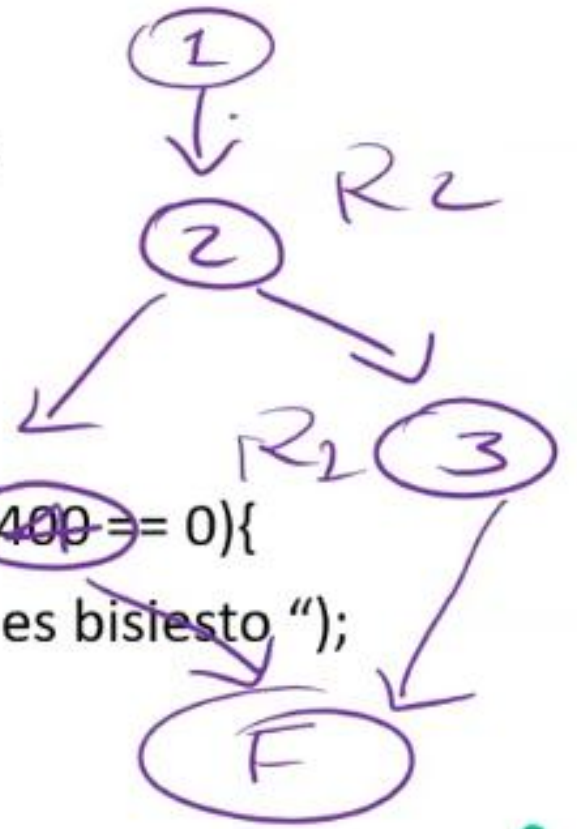


Ejercicios

```
public static void main(String[] args) {  
    Scanner sc=new Scanner(System.in);  
    int a;  
    System.out.println("Ingresa año");  
    a= sc.nextInt();  
    if(a % 4 == 0 && a % 100 != 0 || a % 400 == 0){  
        System.out.println("El año "+a+" Si es bisiesto ");  
    }  
    else{  
        System.out.println("El año "+a+" No es bisiesto ");  
    }  
}
```

Ejercicios

```
public static void main(String[] args) {  
    Scanner sc=new Scanner(System.in);  
    int a;  
    System.out.println("Ingresa año");  
    a= sc.nextInt();  
    if(a % 4 == 0 && a % 100 != 0 || a % 400 == 0){  
        System.out.println("El año "+a+" Si es bisiesto ");  
    }  
    else{  
        System.out.println("El año "+a+" No es bisiesto ");  
    }  
}
```



Ejercicios

```
Import      java.io.*;

Public class Maximo {

    public static void main (String args[]) throws IOException{
        BufferedReader entrada = new BufferedReader (new
        InputStreamReader(System.in));
        Int x,y,z,max;
        System.out.println("Introduce x,y,z: ");
        x = Integer.parseInt (entrada.readLine());
        y = Integer.parseInt (entrada.readLine());
        z = Integer.parseInt (entrada.readLine());
        if (x>y && x>z)
            max = x;
        else
            if (z>y)
                max = z;
            else
                max = y;
        System.out.println ("El máximo es "+ max);
    }
}
```

Ejercicios

```
int sumar(int N, int M){  
    /*Regresa la suma de los datos de la matriz A*/  
    int res = 0;  
    for(int i=0; i<N; i++){  
        for(int j=0; j<M; j++){  
            res += A[i][j];  
        }  
    }  
    return res;  
}
```

Ejercicios

```
int sumar(int N, int M){  
    /*Regresa la suma de los datos de la matriz A*/  
    1 int res = 0;  
    2 for(int i=0; i<N; i++){  
        3 for(int j=0; j<M; j++){  
            4 res += A[i][j];  
        }  
    }  
    5 return res;  
}
```



$$C_c = a - n + 2 = 6 - 5 + 2 = 3$$

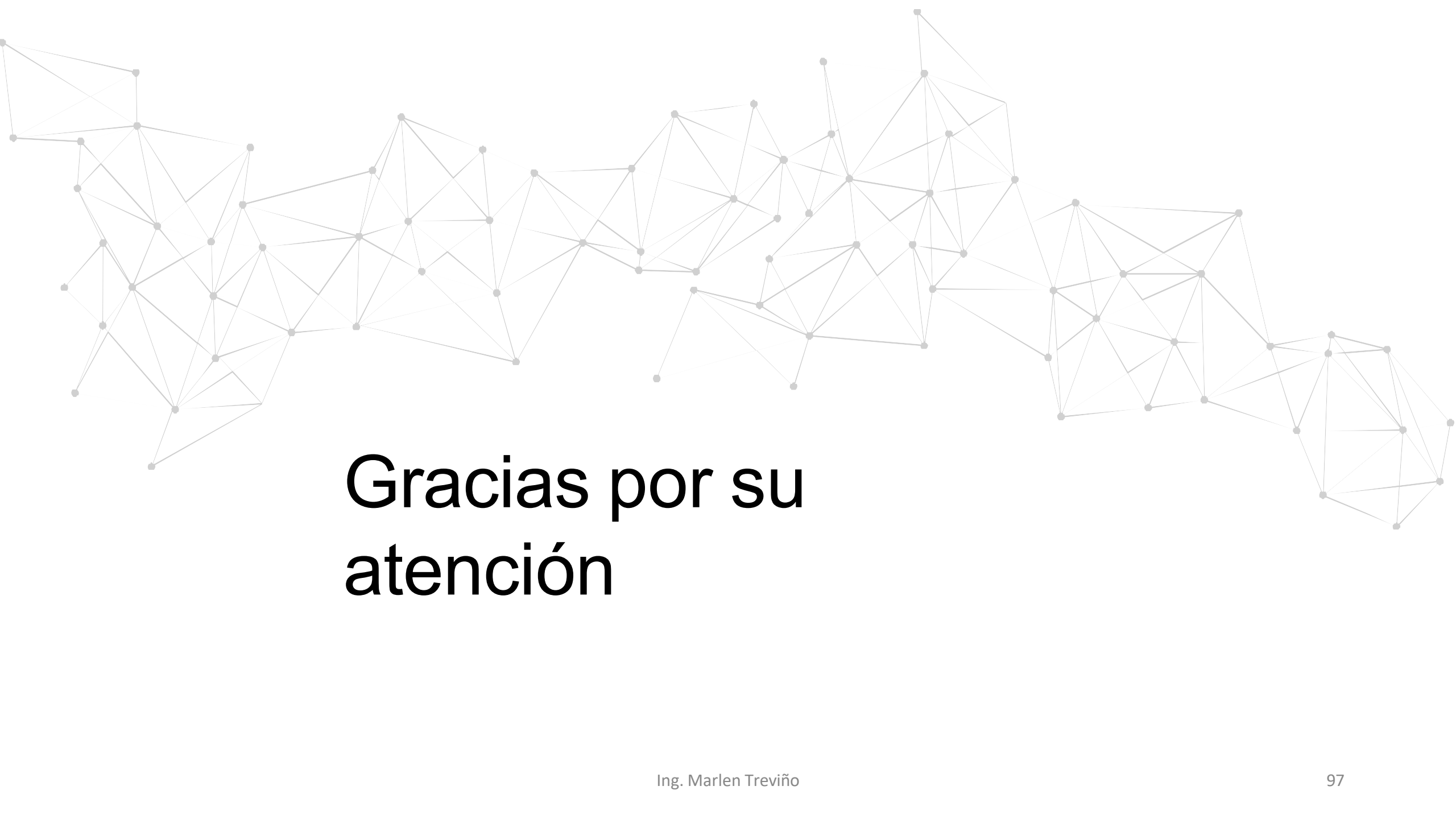
$$C_c = d + 1 = 2 + 1 = 3$$

$$C_c = r = 3$$

$$C_1 = 125$$

$$C_2 = 12325$$

$$C_3 = 123425$$



**Gracias por su
atención**